



Universiteit
Leiden

Machine Learning

The Art and Science of Algorithms that Make Sense of Data

Dr. Mitra Baratchi

13 February 2026

Leiden Institute of Advanced Computer Science - Leiden University

Session 2: Binary classification and related tasks ¹

Reading material

→ Chapter 2, Machine learning: the art and science of algorithms that make sense of data

¹The slides partially include material based on the slides accompanying the book: ***Machine learning: the art and science of algorithms that make sense of data*** Cambridge University Press, 2012.

Table of Contents

1. Binary classification and related tasks
2. Classification
 - Assessing classification performance
 - Visualising classification performance
3. Scoring and ranking
 - Assessing the ranking performance
4. Class probability estimation
 - Assessing class probability estimates

Binary classification and related tasks

Tasks

We are going to take a look at a broad range of machine learning tasks:

- **Tasks:** problems that can be solved with machine learning
- **Instances:** the objects of interest in machine learning described with **features**
- **Instance space:** the space of all possible instances
- **Label space:** labels of example instances $|\mathcal{C}|$
- **Output space:** the space of the outcomes (or target) of the task
- **Model:** a mapping from the input space to the output space potentially using labelled examples

Predictive machine learning scenarios

<i>Task</i>	<i>Label space</i>	<i>Output space</i>	<i>Learning problem</i>
Classification	$\mathcal{L} = \mathcal{C}$ ²	$\mathcal{Y} = \mathcal{C}$	learn an approximation $\hat{c} : \mathcal{X} \rightarrow \mathcal{C}$ ³ to the true labelling function c

² $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

³ $\mathcal{X}$ denotes the input space

Predictive machine learning scenarios

<i>Task</i>	<i>Label space</i>	<i>Output space</i>	<i>Learning problem</i>
Classification	$\mathcal{L} = \mathcal{C}$ ²	$\mathcal{Y} = \mathcal{C}$	learn an approximation $\hat{c} : \mathcal{X} \rightarrow \mathcal{C}$ ³ to the true labelling function c
Scoring and ranking	$\mathcal{L} = \mathcal{C}$	$\mathcal{Y} = \mathbb{R}^{ \mathcal{C} }$	learn a model that outputs a score vector over classes

² $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

³ $\mathcal{X}$ denotes the input space

Predictive machine learning scenarios

<i>Task</i>	<i>Label space</i>	<i>Output space</i>	<i>Learning problem</i>
Classification	$\mathcal{L} = \mathcal{C}^2$	$\mathcal{Y} = \mathcal{C}$	learn an approximation $\hat{c} : \mathcal{X} \rightarrow \mathcal{C}$ ³ to the true labelling function c
Scoring and ranking	$\mathcal{L} = \mathcal{C}$	$\mathcal{Y} = \mathbb{R}^{ \mathcal{C} }$	learn a model that outputs a score vector over classes
Probability estimation	$\mathcal{L} = \mathcal{C}$	$\mathcal{Y} = [0, 1]^{ \mathcal{C} }$	learn a model that outputs a probability vector over classes

² $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

³ $\mathcal{X}$ denotes the input space

Classification

Classification

Classification

A *classifier* is a mapping $\hat{c} : \mathcal{X} \rightarrow \mathcal{C}$, where $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ is a finite and usually small set of *class labels*.

Binary classifier

Binary classifier has only two class labels (e.g., positive and negative, True and False, spam and non-spam).

Classification

Examples of a classifier take the form $(x, c(x))$, where $x \in \mathcal{X}$ is an instance and $c(x)$ is the true class of the instance. $\hat{c}(x)$ is an estimate of the true but unknown function $c(x)$.

Classification

Examples of a classifier take the form $(x, c(x))$, where $x \in \mathcal{X}$ is an instance and $c(x)$ is the true class of the instance. $\hat{c}(x)$ is an estimate of the true but unknown function $c(x)$.

Learning a classifier

Learning a classifier involves constructing the **function** $\hat{c}$ such that it **matches** c as closely as possible.

→ How can you ensure that $\hat{c}$ is matching c ?

Table of content

1. Binary classification and related tasks
2. Classification
 - Assessing classification performance
 - Visualising classification performance
3. Scoring and ranking
 - Assessing the ranking performance
4. Class probability estimation
 - Assessing class probability estimates

Evaluating the performance of classifiers

How to assess the performance of a binary classifier?

- Contingency table
- Various other performance metrics

Evaluating the performance of classifiers

How to assess the performance of a binary classifier?

- **Contingency table** ←
- Various other performance metrics

Contingency table/cross table/confusion matrix

Contingency table

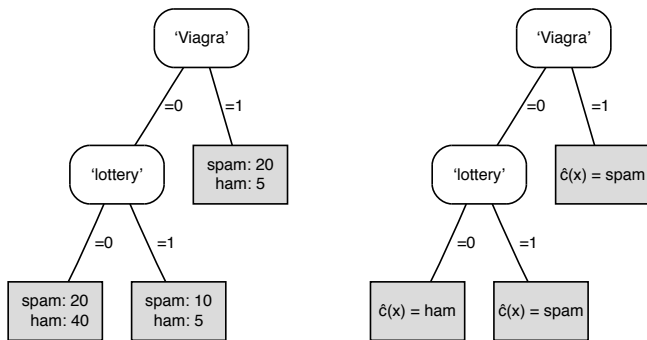
A matrix that displays the (multivariate) frequency distribution of the variables (in this case actual $c(x)$ and predicted $\hat{c}(x)$ variables).

Contingency table

A two-class *contingency table* or *confusion matrix* depicting the performance of the decision tree ($c(x)$ denotes actual and $\hat{c}(x)$ denotes predicted class labels):

	Predicted $\oplus$	Predictive $\ominus$
Actual $\oplus$	True positives	False negatives
Actual $\ominus$	False positives	True negatives

A decision tree



(left) A feature tree with training set class distribution in the leaves. *(right)* A decision tree obtained using the majority class decision rule.

Contingency table

A two-class *contingency table* or *confusion matrix* depicting the performance of a binary classification decision tree (actual $c(x)$ and predicted $\hat{c}(x)$):

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	30	20	50
Actual $\ominus$	10	40	50
	40	60	100

Contingency table

What can we do with this contingency table?

Contingency table

What can we do with this contingency table?

- We can calculate a number of performance metrics

Accuracy and error rate

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

⁴ T_e shows the number of test instances

Accuracy and error rate

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- **Accuracy:** $acc = \frac{1}{|Te|} \sum_{x \in Te} I[\hat{c}(x) = c(x)] = (60 + 15)/100 = 0.75$

⁴ Te shows the number of test instances

Accuracy and error rate

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- **Accuracy:** $acc = \frac{1}{|Te|} \sum_{x \in Te} I[\hat{c}(x) = c(x)] = (60 + 15)/100 = 0.75$
- **Error rate:** $err = \frac{1}{|Te|} \sum_{x \in Te} I[\hat{c}(x) \neq c(x)] = 25/100 = 0.25$

⁴ $|Te|$ shows the number of test instances

Accuracy and error rate

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	99	0	99
Actual $\ominus$	1	0	1
	100	0	100

Imagine that we had an *unbalanced dataset* with 99 actual positives and 1 actual negative. Let's assess the performance of a classifier that predicts everything as positive.

Accuracy and error rate

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	99	0	99
Actual $\ominus$	1	0	1
	100	0	100

Imagine that we had an *unbalanced dataset* with 99 actual positives and 1 actual negative. Let's assess the performance of a classifier that predicts everything as positive.

- *Accuracy*: $acc = (99)/100 = 0.99$
- *Error rate*: $err = 1/100 = 0.01$

Accuracy and error rate

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	99	0	99
Actual $\ominus$	1	0	1
	100	0	100

Imagine that we had an *unbalanced dataset* with 99 actual positives and 1 actual negative. Let's assess the performance of a classifier that predicts everything as positive.

- *Accuracy*: $acc = (99)/100 = 0.99$
- *Error rate*: $err = 1/100 = 0.01$

→ These metrics do not fully evaluate the strength of the classifier (especially if the minority class is interesting).

True/false positive/negative rates

	<i>Predicted</i> $\oplus$	<i>Predicted</i> $\ominus$	
<i>Actual</i> $\oplus$	60	15	75
<i>Actual</i> $\ominus$	10	15	25
	70	30	100

True/false positive/negative rates

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- *True positive rate, sensitivity, recall: rec or*

$$tpr = \frac{\sum_{x \in T_e} I[\hat{c}(x) = c(x) = \oplus]}{\sum_{x \in T_e} I[c(x) = \oplus]} = 60/75 = 0.80$$

True/false positive/negative rates

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- *True positive rate, sensitivity, recall: rec or*

$$tpr = \frac{\sum_{x \in T_e} I[\hat{c}(x) = c(x) = \oplus]}{\sum_{x \in T_e} I[c(x) = \oplus]} = 60/75 = 0.80$$

- *True negative rate, specificity: $spec$ or*

$$tnr = \frac{\sum_{x \in T_e} I[\hat{c}(x) = c(x) = \ominus]}{\sum_{x \in T_e} I[c(x) = \ominus]} = 15/25 = 0.60$$

True/false positive/negative rates

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- *True positive rate, sensitivity, recall: rec or*

$$tpr = \frac{\sum_{x \in T_e} I[\hat{c}(x)=c(x)=\oplus]}{\sum_{x \in T_e} I[c(x)=\oplus]} = 60/75 = 0.80$$

- *True negative rate, specificity: $spec$ or*

$$tnr = \frac{\sum_{x \in T_e} I[\hat{c}(x)=c(x)=\ominus]}{\sum_{x \in T_e} I[c(x)=\ominus]} = 15/25 = 0.60$$

- *False positive rate, false alarm rate: $fpr = 10/25$*

True/false positive/negative rates

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- *True positive rate, sensitivity, recall: rec or*

$$tpr = \frac{\sum_{x \in T_e} I[\hat{c}(x)=c(x)=\oplus]}{\sum_{x \in T_e} I[c(x)=\oplus]} = 60/75 = 0.80$$

- *True negative rate, specificity: $spec$ or*

$$tnr = \frac{\sum_{x \in T_e} I[\hat{c}(x)=c(x)=\ominus]}{\sum_{x \in T_e} I[c(x)=\ominus]} = 15/25 = 0.60$$

- *False positive rate, false alarm rate: $fpr = 10/25$*
- *False negative rate: $fnr = 15/75$*

Precision and recall

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

Precision and recall

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- *Precision, confidence:* $prec = \frac{\sum_{x \in Te} I[\hat{c}(x)=c(x)=\oplus]}{\sum_{x \in Te} I[\hat{c}(x)=\oplus]} = 60/70$

Precision and recall

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

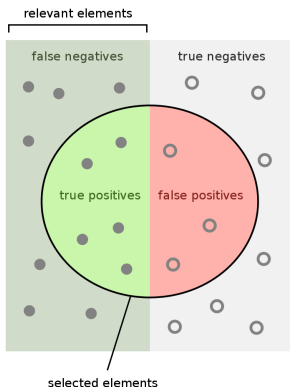
- *Precision, confidence*: $prec = \frac{\sum_{x \in Te} I[\hat{c}(x)=c(x)=\oplus]}{\sum_{x \in Te} I[\hat{c}(x)=\oplus]} = 60/70$

- *True positive rate, sensitivity, recall*: rec or

$$tpr = \frac{\sum_{x \in Te} I[\hat{c}(x)=c(x)=\oplus]}{\sum_{x \in Te} I[c(x)=\oplus]} = 60/75 = 0.80$$

→ In this case the positive class is the most relevant class

Sensitivity/specificity *vs* Precision/recall



How many relevant items are selected?
e.g. How many sick people are correctly identified as having the condition.

$$\text{Sensitivity} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

How many negative selected elements are truly negative?
e.g. How many healthy people are identified as not having the condition.

$$\text{Specificity} = \frac{\text{true negatives}}{\text{true negatives} + \text{false positives}}$$

How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

F-measure

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

F-measure

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- *F-measure or F_1 score*: $F_1 = \frac{2}{\frac{1}{prec} + \frac{1}{rec}} = 2 \cdot \frac{prec \cdot rec}{prec + rec}$

F-measure

F-measure is a performance metric that is not affected by negatives, as a weighted harmonic mean of precision and recall.

F-measure

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	60	15	75
Actual $\ominus$	10	15	25
	70	30	100

- *F-measure or F_1 score*: $F_1 = \frac{2}{\frac{1}{prec} + \frac{1}{rec}} = 2 \cdot \frac{prec \cdot rec}{prec + rec}$

F-measure

F-measure is a performance metric that is not affected by negatives, as a weighted harmonic mean of precision and recall.

→ The harmonic mean is used when we are interested in an average of ratios (i.e., rates).

Evaluation approach

How to use these metrics such that they can represent how well the method performs during the inference phase? → Does the model *generalise*?

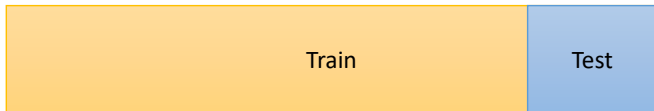
Evaluation approach

How to use these metrics such that they can represent how well the method performs during the inference phase? → Does the model *generalise*?

Train/test splits

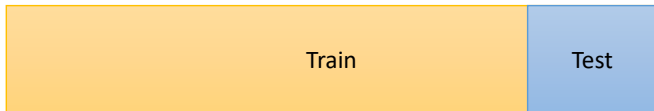
It is common to use separate *train-test* splits to assess the performance of machine learning algorithms. The test set is used to make predictions during the *inference phase* on data not used during the *training phase*.

Training and validation



During training we simulate the inference phase → we reserve part of the data we have for testing the algorithm.

Training and validation

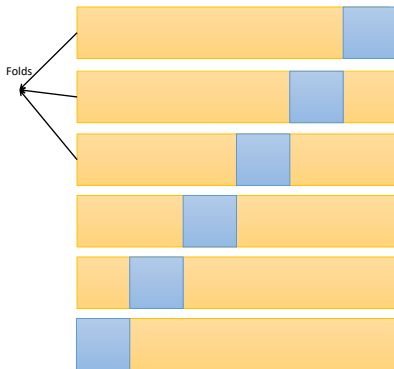


During training we simulate the inference phase → we reserve part of the data we have for testing the algorithm.

→ The problem is that we do not know if the data during inference is actually like the test set we reserved.

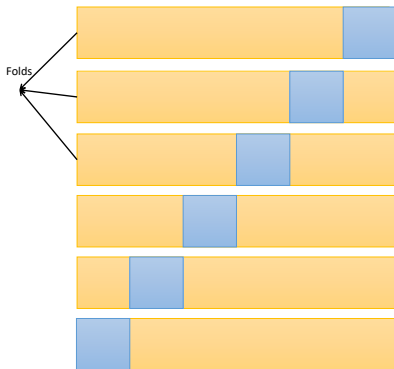
Cross-validation

We can repeat this process on different parts (using *folds*) of data:



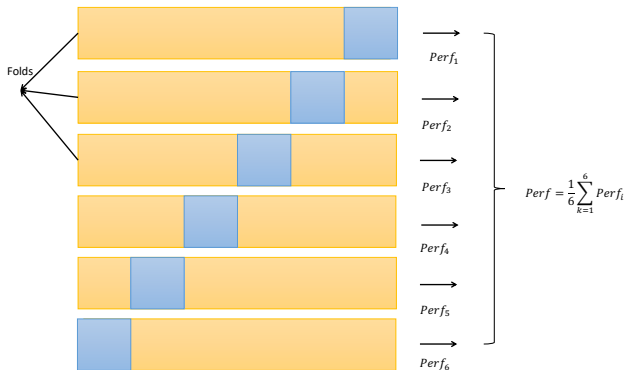
Cross-validation

We can repeat this process on different parts (using *folds*) of data:



Finally, we need a way to aggregate the performances

Cross-validation



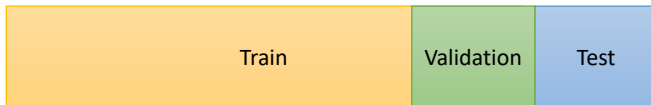
Evaluation approaches

Sometimes algorithms have many knobs to tune (i.e., hyper-parameters) per dataset. Furthermore, we might also have many algorithms to compare.

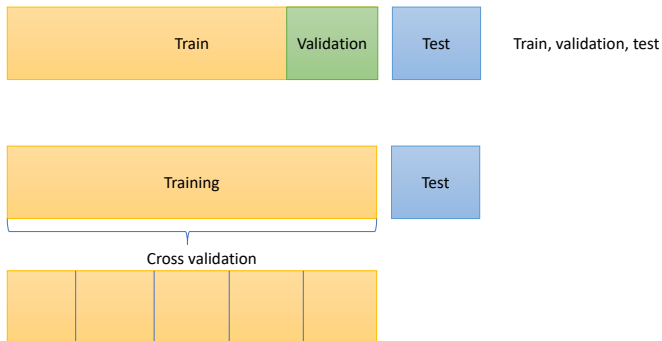
If you use the train/test splits for all these comparisons you might get a model biased to the train/test dataset at hand.

We can use a separate *validation* set for these intermediate tasks.

Training, validation, test



Training, validation, tests



Over/under-fitting

- *Over-fitting*: is a situation where a model performs very well on the training data by learning the detail and noise in the training data while performing very poorly on the test data.
- *Under-fitting*: is a situation where a model performs poorly both on the training and test data.

Over-fitting and generalisability

Both over-fitting and under-fitting show low *generalisability*.

Performance measures

`https://colab.research.google.com/drive/
1DD2SeUBOePI3m8ijSMccEu6SqyumcfC1?usp=sharing`

Table of content

1. Binary classification and related tasks
2. Classification
 - Assessing classification performance
 - Visualising classification performance
3. Scoring and ranking
 - Assessing the ranking performance
4. Class probability estimation
 - Assessing class probability estimates

Visualising the performance of the binary classifier

Let's take a look at a number of useful tools to visualise the performance of binary classifiers...

Decision rule for a binary classifier

Imagine that our *decision rule* for the binary classification on

$D = \{(x_i, y_i)\}$ is $\hat{c}(x) = I(f(x) > \tau)^6 \rightarrow$

This decision rule, τ , is important because it can lead to different classification results.

- For each given value of τ , we can apply our decision rule, create a confusion matrix and count the number of true positives, false positives, true negatives, and false negatives that occur

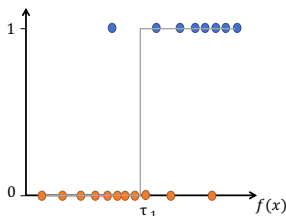
⁶ I in this case is the indicator function

Contingency table for τ_1

	<i>Predicted</i> $\oplus$	<i>Predicted</i> $\ominus$	
<i>Actual</i> $\oplus$	7	1	8
<i>Actual</i> $\ominus$	3	8	11
	10	9	19

Sensitivity $\rightarrow 7/8$

Specificity $\rightarrow 8/11$

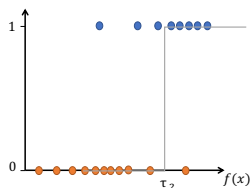


Contingency table for τ_2

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	5	3	8
Actual $\ominus$	1	10	11
	6	13	19

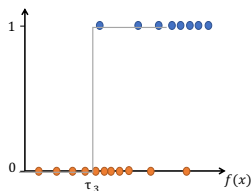
Sensitivity $\rightarrow 5/8$

Specificity $\rightarrow 10/11$



Contingency table for τ_3

	Predicted $\oplus$	Predicted $\ominus$	
Actual $\oplus$	8	0	8
Actual $\ominus$	7	4	11
	15	4	19

Sensitivity $\rightarrow 8/8$ Specificity $\rightarrow 4/11$ 

For this specific example, we can have many more confusion matrices

Visualising the performance of a binary classifier

For this specific example, we can have many more confusion matrices by changing the value of τ ...

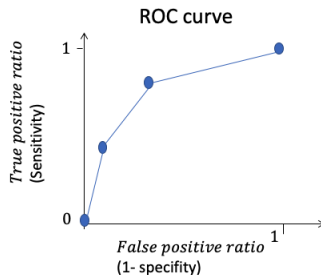
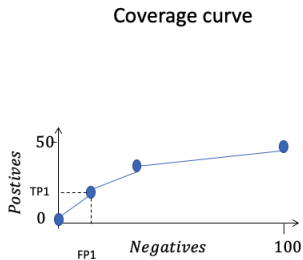
→ how can we summarise all these matrices?

ROC curve and coverage plot

ROC curve and coverage plot

A *coverage plot* and a *Receiver Operation Curve (ROC)* (normalized coverage plot) both help summarise all different confusion matrices.

ROC plot and coverage plot

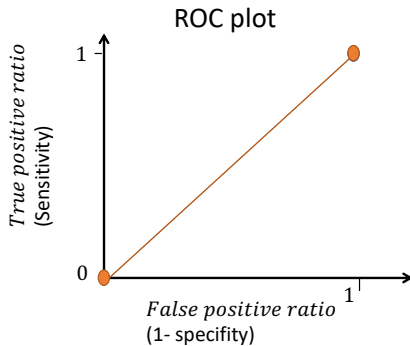


(left) coverage plot (right) ROC plot

ROC plot and coverage plot

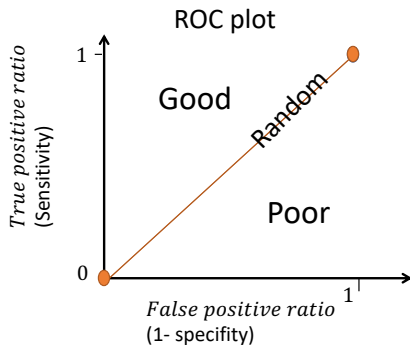
How should we know if a classifier is good or bad based on its ROC curve? → Let us see what a no-skill classifier looks like.

ROC plot



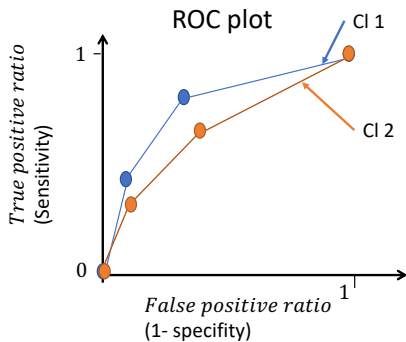
ROC plot of a no skill classifier

ROC plot of a no-skill classifier



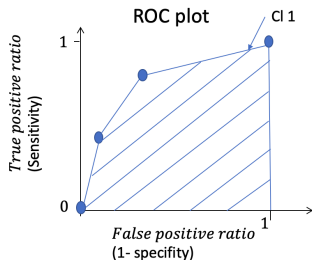
ROC plot of a no-skill classifier

Comparing the performance of classifiers using ROC curves



Comparing the ROC plots of two classifiers.

Area under the curve of the ROC plot



Area under the curve.

Area under the curve

The *Area Under the Curve (AUC)* can be used as a summary of the model skills.

- The curves of different models can be compared directly in general or for different thresholds.

Scoring and ranking

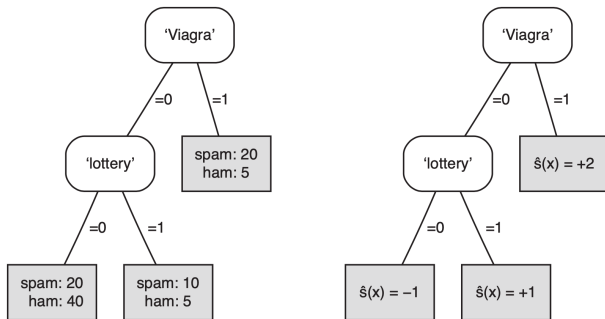
Scoring classifier

scoring classifier

A *scoring classifier* is a mapping $\hat{\mathbf{s}}: \mathcal{X} \rightarrow \mathbb{R}^k$, i.e., a mapping from the instance space to a k -vector of real numbers.

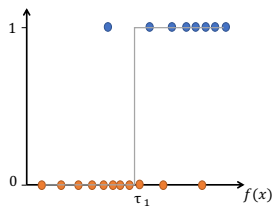
- $\hat{s}_i(x)$ is the score assigned to class C_i for instance x
- This score indicates how likely it is that class label C_i applies.
- If we only have two classes, it usually suffices to consider the score for only one of the classes; in that case, we use $\hat{s}(x)$ to denote the score of the positive class for instance x .

Scoring tree

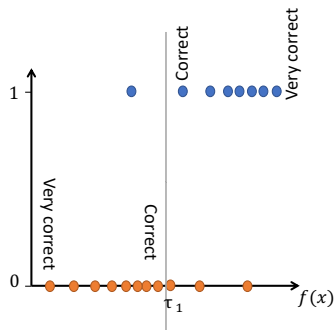


(left) A feature tree with training set class distribution in the leaves. (right) A scoring tree using the logarithm of the class ratio as scores; spam is taken as the positive class. Scores are acquired by $\log_2$ of the class ratio.

Scoring classifier

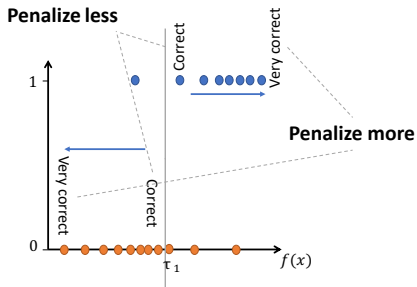


Scoring classifier



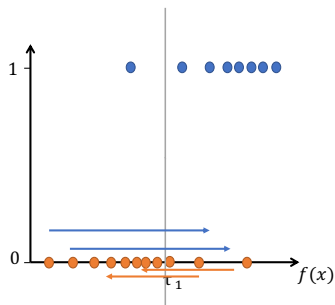
We want a classifier that makes a distinction between very correct and correct cases.

Scoring classifier



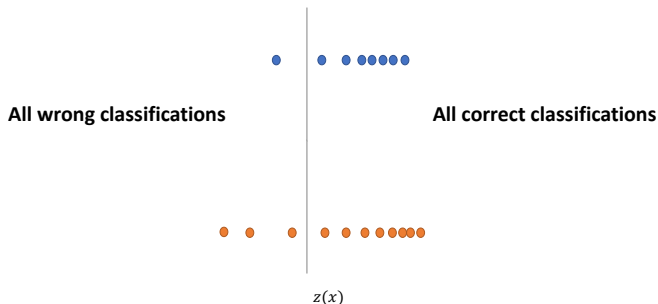
We want a classifier that makes a distinction between very correct and correct cases using scores. → One way to achieve this is to penalize the classifier based on any misclassification

Scoring classifier



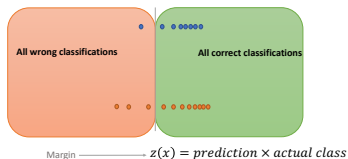
Let us move all the correct classifications on one side and wrong classifications on the other side

Scoring classifier



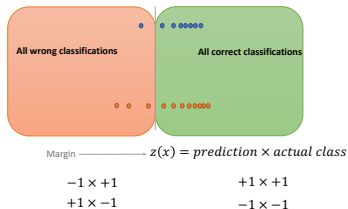
The line separating the correct and wrong classifications is called the *margin* of the classifier.

Margin of the classifier



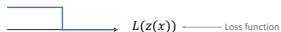
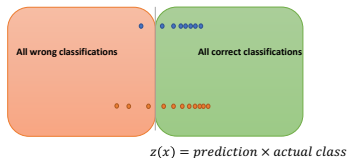
- **Margin:** If we take the true class $c(x)$ as $+1$ for positive examples and -1 for negative examples, then the quantity $z(x) = c(x)\hat{s}(x)$ is positive for correct predictions and negative for incorrect predictions.

Margin of the classifier



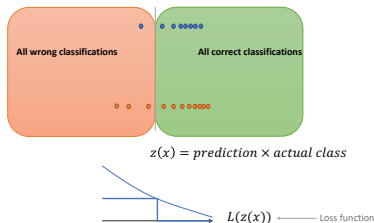
- Margin:** If we take the true class $c(x)$ as $+1$ for positive examples and -1 for negative examples, then the quantity $z(x) = c(x)\hat{s}(x)$ is positive for correct predictions and negative for incorrect predictions

Loss function



- **Loss function** $L : \mathbb{R} \mapsto [0, \infty)$ maps each example's margin $z(x)$ to an associated loss $L(z(x))$. It rewards large positive margins and penalizes large negative values.
- The average loss over a test set Te is $\frac{1}{|Te|} \sum_{x \in Te} L(z(x))$.

Loss function

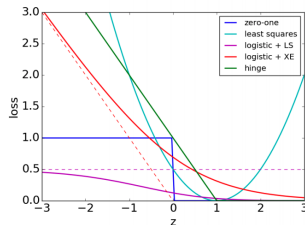


- **Loss function** $L : \mathbb{R} \mapsto [0, \infty)$ which maps each example's margin $z(x)$ to an associated loss $L(z(x))$. It rewards large positive margins and penalizes large negative values.
- The average loss over a test set Te is $\frac{1}{|Te|} \sum_{x \in Te} L(z(x))$.

Loss functions

<i>Loss</i>	<i>Definition</i>
0-1	$L_{01}(z) = 1$ if $z \leq 0$, $L_{01}(z) = 0$ if $z > 0$
Hinge	$L_h(z) = (1 - z)$ if $z \leq 1$
Exponential	$L_{\exp}(z) = \exp(-z)$
Logistic	$L_{\log}(z) = \log_2(1 + \exp(-z))$
Squared	$L_{\text{sq}}(z) = (1 - z)^2$

Loss functions



Different loss functions used by machine learning classifiers

⁷ → the squared loss can be set to 0 for $z > 1$

⁷ Image source: https://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec4.pdf

Average loss

- The average loss over a test set Te is $\frac{1}{|Te|} \sum_{x \in Te} L(z(x))$.
- We should aim to find the margin that helps minimise the average loss
- Most machine learning techniques work by minimising such a loss function using an optimisation technique

Table of content

1. Binary classification and related tasks
2. Classification
 - Assessing classification performance
 - Visualising classification performance
3. Scoring and ranking
 - Assessing the ranking performance
4. Class probability estimation
 - Assessing class probability estimates

Ranking error

Imagine that we have two scoring classifiers and we want to compare them. We can use the scores to create rankings for instances → how can we assess how good these rankings are?

Ranking error

The *ranking error rate* is defined as

$$\text{rank-err} = \frac{\sum_{x \in Te^{\oplus}, x' \in Te^{\ominus}} I[\hat{s}(x) < \hat{s}(x')] + \frac{1}{2} I[\hat{s}(x) = \hat{s}(x')]}{Pos \cdot Neg}$$

→ For any pair of positive and negative examples we penalise if the score of the positive example is lower than the score of the negative example.

Class probability estimation

Class probability estimation

Probability estimator

A class *probability estimator* is a scoring classifier that outputs probability vectors over classes, i.e., a mapping $\hat{\mathbf{p}} : \mathcal{X} \rightarrow [0, 1]^k$. We write $\hat{\mathbf{p}}(x) = (\hat{p}_1(x), \dots, \hat{p}_k(x))$, where $\hat{p}_i(x)$ is the probability assigned to class C_i for instance x , and $\sum_{i=1}^k \hat{p}_i(x) = 1$.

- The output of the classifier shows how likely it is that the instance belongs to a specific class rather than which class it will belong to.
- If we have only two classes, the probability associated with one class is 1 minus the probability of the other class; in that case, we use $\hat{p}(x)$ to denote the estimated probability of the positive class for instance x .

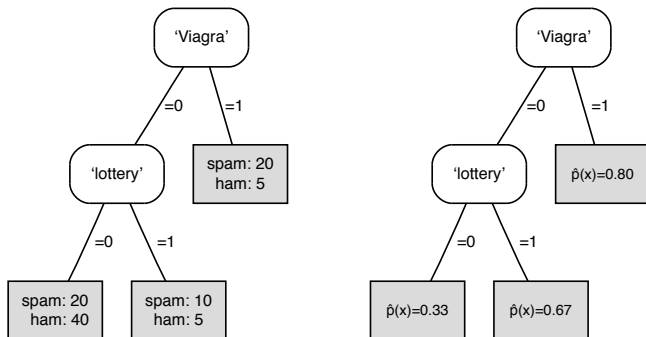
Class probability estimation

Probability estimator

A class *probability estimator* is a scoring classifier that outputs probability vectors over classes, i.e., a mapping $\hat{\mathbf{p}} : \mathcal{X} \rightarrow [0, 1]^k$. We write $\hat{\mathbf{p}}(x) = (\hat{p}_1(x), \dots, \hat{p}_k(x))$, where $\hat{p}_i(x)$ is the probability assigned to class C_i for instance x , and $\sum_{i=1}^k \hat{p}_i(x) = 1$.

- How do we calculate probabilities?

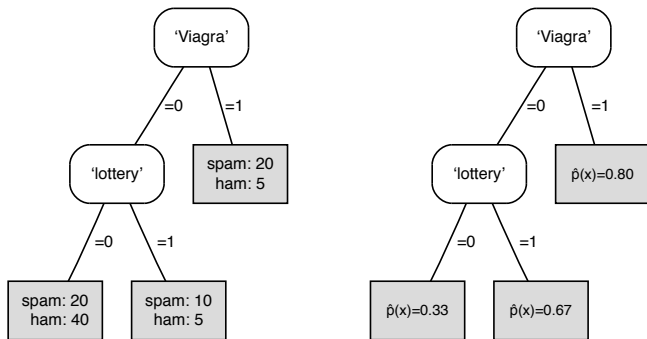
A decision tree



(left) A feature tree with training set class distribution in the leaves. *(right)* the probability estimation for the same tree.

Imagine that the threshold is 0.5

A decision tree



(left) A feature tree with training set class distribution in the leaves. *(right)* the probability estimation for the same tree.

Imagine that the threshold is 0.5 $\rightarrow$ this is equivalent to the majority class

Class probabilities


[Install](#) [User Guide](#) [API](#) [Examples](#) [More ▾](#)

[Prev](#) [Up](#) [Next](#)

scikit-learn 0.24.0
[Other versions](#)

Please [cite us](#) if you use the software.

Search Results

Search finished, found 56 page(s) matching the search query.

- [sklearn.calibration.CalibratedClassifierCV.predict_proba](#)
- [sklearn.discriminant_analysis.LinearDiscriminantAnalysis.predict_proba](#)
- [sklearn.discriminant_analysis.QuadraticDiscriminantAnalysis.predict_proba](#)
- [sklearn.dummy.DummyClassifier.predict_proba](#)
- [sklearn.ensemble.AdaBoostClassifier.predict_proba](#)
- [sklearn.ensemble.AdaBoostClassifier.staged_predict_proba](#)
- [sklearn.ensemble.BaggingClassifier.predict_proba](#)
- [sklearn.ensemble.ExtraTreesClassifier.predict_proba](#)
- [sklearn.ensemble.GradientBoostingClassifier.predict_proba](#)
- [sklearn.ensemble.GradientBoostingClassifier.staged_predict_proba](#)
- [sklearn.ensemble.HistGradientBoostingClassifier.predict_proba](#)
- [sklearn.ensemble.HistGradientBoostingClassifier.staged_predict_proba](#)
- [sklearn.ensemble.RandomForestClassifier.predict_proba](#)
- [sklearn.ensemble.StackingClassifier.predict_proba](#)
- [sklearn.ensemble.VotingClassifier.predict_proba](#)
- [sklearn.feature_selection.RFE.predict_proba](#)
- [sklearn.feature_selection.RFECV.predict_proba](#)
- [sklearn.gaussian_process.GaussianProcessClassifier.predict_proba](#)
- [sklearn.linear_model.LogisticRegression.predict_proba](#)
- [sklearn.linear_model.LogisticRegressionCV.predict_proba](#)
- [sklearn.linear_model.SGDClassifier.predict_proba](#)
- [sklearn.mixture.BayesianGaussianMixture.predict_proba](#)
- [sklearn.mixture.GaussianMixture.predict_proba](#)

Toggle Menu

Table of content

1. Binary classification and related tasks
2. Classification
 - Assessing classification performance
 - Visualising classification performance
3. Scoring and ranking
 - Assessing the ranking performance
4. Class probability estimation
 - Assessing class probability estimates

Mean squared probability error

We need a way to assess how good these probabilities are.

Squared error

The *squared error (SE) of the predicted probability vector*

$\hat{\mathbf{p}}(x) = (\hat{p}_1(x), \dots, \hat{p}_k(x))$, where I is a binary vector which has the i -th bit set to 1 if x 's true class is C_i , is defined as

$$\text{SE}(x) = \frac{1}{2} \sum_{i=1}^k (\hat{p}_i(x) - I[c(x) = C_i])^2$$

and the *mean squared error (MSE)* as the average squared error over all instances in the test set:

$$\text{MSE}(Te) = \frac{1}{|Te|} \sum_{x \in Te} \text{SE}(x)$$

Lessons learned

- Different variants of classification
 - Classification
 - Scoring and ranking
 - Probability estimation
- Validating the performance of classifiers
 - Classification: contingency tables, different performance metrics, train/test/validation, ROC curves
 - Scoring and ranking: ranking error
 - Probability estimation: probability estimation error